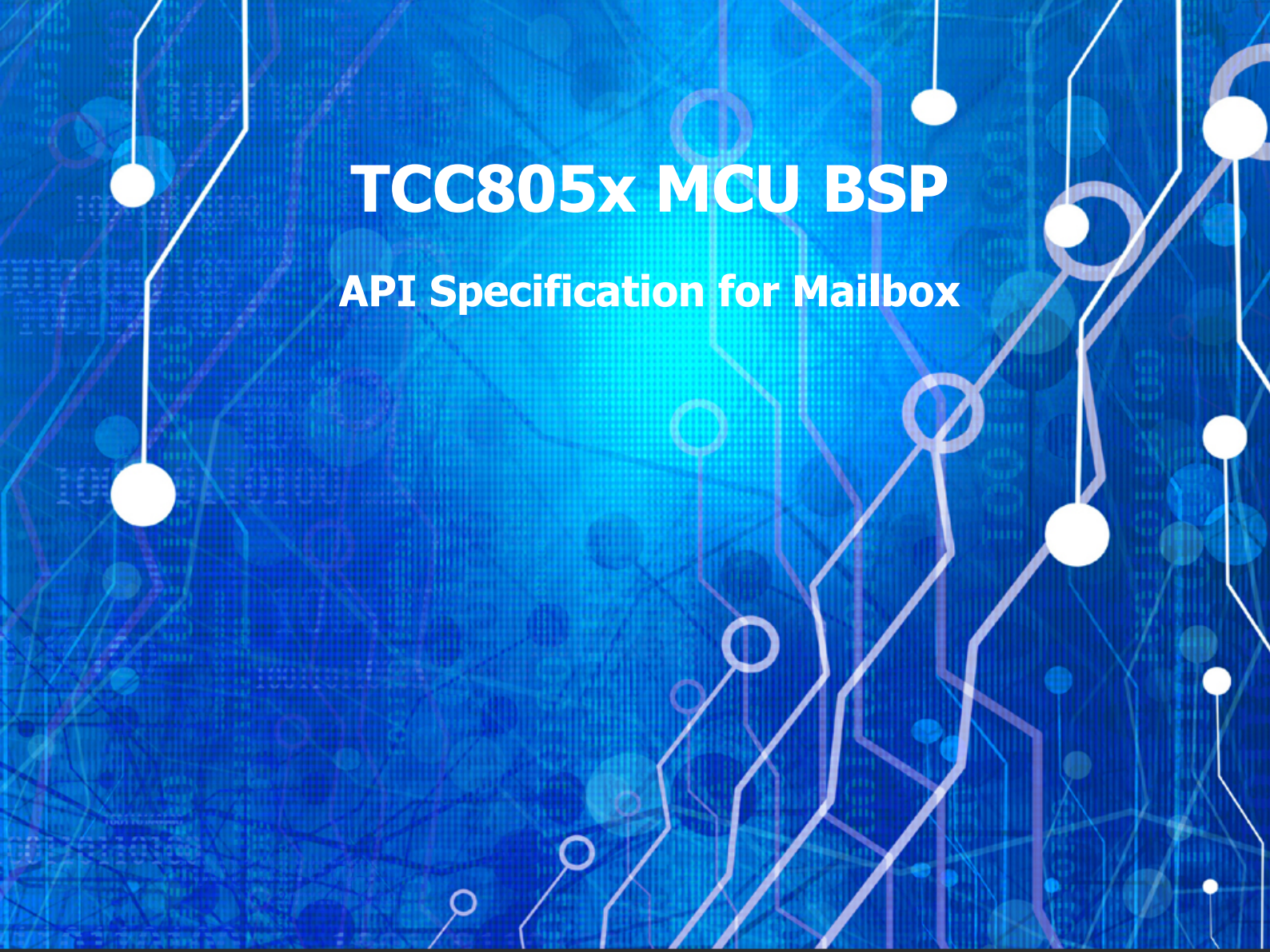




TCC805x MCU BSP

API Specification for Mailbox

Rev. 1.01 [A]
2021-06-08

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files.....	4
4	Enumerations	5
4.1	MBOXCh_t	5
5	Constants.....	6
5.1	Mailbox I/O Commands	6
5.2	Mailbox Error Defines.....	8
5.3	Mailbox Device Error Defines	10
5.4	Others.....	11
6	Structures	12
6.1	MBOXCallback_t	12
6.2	MBOXMsage_t.....	14
6.3	MBOXQueue_t.....	15
6.4	MBOXDvice_t	16
7	Callback Functions	17
7.1	MBOXInitFunc	17
7.2	MBOXCallFunc	18
8	APIs	19
8.1	MBOX_Init	19
8.2	MBOX_DevInit.....	20
8.3	MBOX_DevOpen	21
8.4	MBOX_DevSend	22
8.5	MBOX_DevSendSss.....	23
8.6	MBOX_DevSendCmd	24
8.7	MBOX_DevSendData.....	25
8.8	MBOX_DevRecv.....	26
8.9	MBOX_DevRecvSss	27
8.10	MBOX_DevRecvCmd	28
8.11	MBOX_DevRecvData	29
8.12	MBOX_DevIoctl	30
9	How to use Mailbox Driver	31
9.1	Echo Server Example: Blocking Read/Write	31
9.2	Echo Client Example: Non-blocking Read/Write	32
9.3	Emergency Message Transmission Example: Blocking Read/Write	33
9.4	Checking Peer Mailbox Ready Example	34
9.5	Communication With U-Boot Layer of AP	35
10	References	36
11	Revision History	37
	Rev. 1.01: 2021-06-08	37
	Rev. 1.00: 2020-11-12	37

Figures

Figure 3.1 Location of source files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the mailbox device driver (hereinafter referred to as mailbox driver) of TCC805x MCU BSP. For more detailed information of the mailbox in the TCC805x MCU Sub-system, refer to Chapter 18 Mailbox of the "*PART 11-MICOM SUB-SYSTEM*" in the "*TCC805x Full Specification*". [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

MCU	Micro Controller Unit
RTOS	Real Time Operating System
BSP	Board Support Package
API	Application Programmable Instruction
FIFO	First In First Out
SAL	System Abstract Layer

3 SOURCE FILES

3.1 Location of Source Files

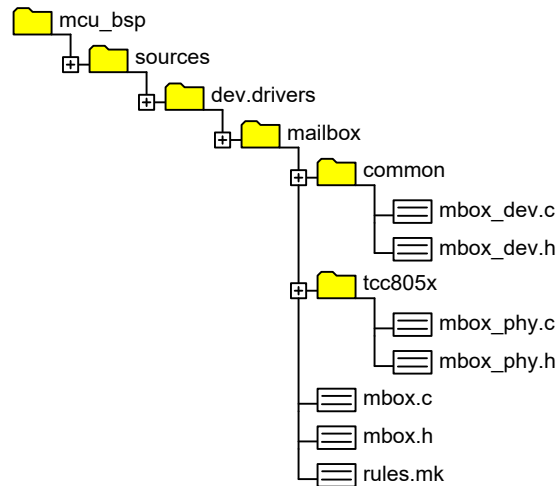


Figure 3.1 Location of source files

3.2 Simple Description of Source Files

- **mbox_dev.c**
 - Source code for user APIs and common software implementation which is not related to hardware.
- **mbox_dev.h**
 - Header file for pre-defined values to control mailbox hardware.
- **mbox_phy.c**
 - Source code for hardware specified implementation.
- **mbox_phy.h**
 - Header file for pre-defined hardware information and specific registers.
- **mbox.c**
 - Source code for registering blocking read/write and setting thread safe configurations.
- **mbox.h**
 - Header file for pre-defined values and user configurations. User APIs are declared in this header file.
- **rules.mk**
 - Build options and target information for mailbox driver.

4 ENUMERATIONS

4.1 MBOXCh_t

Enumeration variables for mailbox communication channels. Every non-secure channel can be used by default. And secure channels can be used if **NS** bit is reset in **SCR_EL3** register of Cortex-R5 CPU. **SCR_EL3** register can be accessed on bootloader.

Declaration

```
typedef enum __MBOX_CHANNEL__
{
    MBOX_CH_INVALID          = -1,
    MBOX_CH_CA53MP_SECURE,
    MBOX_CH_CA53MP_NONSECURE,
    MBOX_CH_CA72MP_SECURE,
    MBOX_CH_CA72MP_NONSECURE,
    MBOX_CH_CM4_NONSECURE,
    MBOX_CH_HSM_NONSECURE,
    MBOX_CH_MAX,
} MBOXCh_t;
```

Description

Value	Description
MBOX_CH_INVALID	For checking validation
MBOX_CH_CA53MP_SECURE	For communication with CA53 secure Mailbox
MBOX_CH_CA53MP_NONSECURE	For communication with CA53 non-secure Mailbox
MBOX_CH_CA72MP_SECURE	For communication with CA72 secure Mailbox
MBOX_CH_CA72MP_NONSECURE	For communication with CA72 non-secure Mailbox
MBOX_CH_CM4_NONSECURE	For communication with CM4 non-secure Mailbox
MBOX_CH_HSM_NONSECURE	For communication with HSM non-secure Mailbox
MBOX_CH_MAX	For checking validation

Remark

None

5 CONSTANTS

5.1 Mailbox I/O Commands

Mailbox device driver provides I/O commands to set and reset the mailbox hardware directly. You can control mailbox hardware and check mailbox hardware status.

Declaration

```
#define MBOX_IO_CMD_INVALID          (-1)
#define MBOX_IO_CMD_SET_CTRL_REG    (0)
#define MBOX_IO_CMD_CLR_CTRL_REG    (1)
#define MBOX_IO_CMD_GET_ILVL_BIT    (2)
#define MBOX_IO_CMD_GET_INEN_BIT    (3)
#define MBOX_IO_CMD_GET_WOEN_BIT    (4)
#define MBOX_IO_CMD_GET_CFLS_BIT    (5)
#define MBOX_IO_CMD_GET_DFLS_BIT    (6)
#define MBOX_IO_CMD_GET_RCNT_CMD    (7)
#define MBOX_IO_CMD_GET_RFUL_CMD    (8)
#define MBOX_IO_CMD_GET_REMT_CMD    (9)
#define MBOX_IO_CMD_GET_WCNT_CMD    (10)
#define MBOX_IO_CMD_GET_WFUL_CMD    (11)
#define MBOX_IO_CMD_GET_WEMT_CMD    (12)
#define MBOX_IO_CMD_GET_RCNT_DAT    (13)
#define MBOX_IO_CMD_GET_RFUL_DAT    (14)
#define MBOX_IO_CMD_GET_REMT_DAT    (15)
#define MBOX_IO_CMD_GET_WCNT_DAT    (16)
#define MBOX_IO_CMD_GET_WFUL_DAT    (17)
#define MBOX_IO_CMD_GET_WEMT_DAT    (18)
#define MBOX_IO_CMD_GET_OWN_STS     (19)
#define MBOX_IO_CMD_GET_OPP_STS     (20)
#define MBOX_IO_CMD_MAX             (21)
```

Description

Value	Description
<i>MBOX_IO_CMD_INVALID</i>	<i>For checking validation</i>
<i>MBOX_IO_CMD_SET_CTRL_REG</i>	<i>For writing specific bits into MBOX_CTRL Register</i>
<i>MBOX_IO_CMD_CLR_CTRL_REG</i>	<i>For clearing specific bits of MBOX_CTRL Register</i>
<i>MBOX_IO_CMD_GET_ILVL_BIT</i>	<i>For reading ILEVEL [1:0] of MBOX_CTRL Register</i>
<i>MBOX_IO_CMD_GET_INEN_BIT</i>	<i>For reading IEN_READ [4:4] of MBOX_CTRL Register</i>
<i>MBOX_IO_CMD_GET_WOEN_BIT</i>	<i>For reading IEN_WRITE [20:20] of MBOX_CTRL Register</i>
<i>MBOX_IO_CMD_GET_CFLS_BIT</i>	<i>For reading CF_FLUSH [6:6] of MBOX_CTRL Register</i>
<i>MBOX_IO_CMD_GET_DFLS_BIT</i>	<i>For reading DF_FLUSH [7:7] of MBOX_CTRL Register</i>
<i>MBOX_IO_CMD_GET_RCNT_CMD</i>	<i>For reading SCOUNT [23:20] of CMD_FIFO_STS Register</i>
<i>MBOX_IO_CMD_GET_RFUL_CMD</i>	<i>For reading SFUL [17:17] of CMD_FIFO_STS Register</i>
<i>MBOX_IO_CMD_GET_REMT_CMD</i>	<i>For reading SEMP [16:16] of CMD_FIFO_STS Register</i>
<i>MBOX_IO_CMD_GET_WCNT_CMD</i>	<i>For reading MCOUNT [7:4] of CMD_FIFO_STS Register</i>
<i>MBOX_IO_CMD_GET_WFUL_CMD</i>	<i>For reading MFUL [1:1] of CMD_FIFO_STS Register</i>
<i>MBOX_IO_CMD_GET_WEMT_CMD</i>	<i>For reading MEMP [0:0] of CMD_FIFO_STS Register</i>
<i>MBOX_IO_CMD_GET_RCNT_DAT</i>	<i>For reading SCOUNT [15:0] of DAT_FIFO_RX_STS Register</i>
<i>MBOX_IO_CMD_GET_RFUL_DAT</i>	<i>For reading SFUL [30:30] of DAT_FIFO_RX_STS Register</i>
<i>MBOX_IO_CMD_GET_REMT_DAT</i>	<i>For reading SEMP [31:31] of DAT_FIFO_RX_STS Register</i>
<i>MBOX_IO_CMD_GET_WCNT_DAT</i>	<i>For reading MCOUNT [15:0] of DAT_FIFO_TX_STS Register</i>
<i>MBOX_IO_CMD_GET_WFUL_DAT</i>	<i>For reading MFUL [30:30] of DAT_FIFO_TX_STS Register</i>
<i>MBOX_IO_CMD_GET_WEMT_DAT</i>	<i>For reading MEMP [31:31] of DAT_FIFO_TX_STS Register</i>
<i>MBOX_IO_CMD_GET_OWN_STS</i>	<i>For reading OWN_TMN_STS [0:0] TMN_STS Register</i>
<i>MBOX_IO_CMD_GET_OPP_STS</i>	<i>For reading OPP_TMN_STS [16:16] TMN_STS Register</i>
<i>MBOX_IO_CMD_MAX</i>	<i>For checking validation</i>

Remark

None

5.2 Mailbox Error Defines

These error messages are caused by mailbox hardware implementation. These errors do not mean the problem of mailbox hardware itself, but they mean that there are some problems to use mailbox hardware.

Declaration

```
#define MBOX_SUCCESS            (0)
#define MBOX_ERR_COMMON        (-1)
#define MBOX_ERR_BUSY          (-2)
#define MBOX_ERR_NOT_INITIALIZE (-3)
#define MBOX_ERR_TIMEOUT        (-4)
#define MBOX_ERR_WRITE          (-5)
#define MBOX_ERR_READ           (-6)
#define MBOX_ERR_ARGUMENT       (-7)
#define MBOX_ERR_AGAIN          (-8)
#define MBOX_ERR_QUEUE_FULL     (-9)
#define MBOX_ERR_QUEUE_EMPTY    (-10)
#define MBOX_ERR_NO_SUCH_DEV     (-11)
#define MBOX_ERR_FIFO_FULL      (-12)
#define MBOX_ERR_FIFO_EMPTY     (-13)
#define MBOX_ERR_FIFO_SIZE      (-14)
#define MBOX_ERR_MULTI_OPEN     (-15)
#define MBOX_ERR_DEV_NOT_OPENED (-16)
#define MBOX_ERR_DEV_NOT_FOUND  (-17)
#define MBOX_ERR_OS_INIT        (-18)
#define MBOX_ERR_SIG_INIT       (-19)
#define MBOX_ERR_OPP_INIT       (-20)
#define MBOX_ERR_IOCTL          (-21)
```

Description

Value	Description
<i>MBOX_SUCCESS</i>	<i>For success return</i>
<i>MBOX_ERR_COMMON</i>	<i>Reserved</i>
<i>MBOX_ERR_BUSY</i>	<i>Reserved</i>
<i>MBOX_ERR_NOT_INITIALIZE</i>	<i>Error when mailbox hardware is not registered.</i>
<i>MBOX_ERR_TIMEOUT</i>	<i>Error when timeout occurs while Read/Write is blocked.</i>
<i>MBOX_ERR_WRITE</i>	<i>Error when writing data into FIFO register fails.</i>
<i>MBOX_ERR_READ</i>	<i>Error when reading data from FIFO register fails.</i>
<i>MBOX_ERR_ARGUMENT</i>	<i>Error when invalid parameters are passed into driver API.</i>
<i>MBOX_ERR_AGAIN</i>	<i>Reserved</i>
<i>MBOX_ERR_QUEUE_FULL</i>	<i>Error when there are no spaces for received data in user buffer.</i>
<i>MBOX_ERR_QUEUE_EMPTY</i>	<i>Error when there are no data to be read in user buffer.</i>
<i>MBOX_ERR_NO_SUCH_DEV</i>	<i>Error when user buffer is null for received device name.</i>
<i>MBOX_ERR_FIFO_FULL</i>	<i>Error when mailbox FIFO register is full.</i>
<i>MBOX_ERR_FIFO_EMPTY</i>	<i>Error when mailbox FIFO register is empty.</i>
<i>MBOX_ERR_FIFO_SIZE</i>	<i>Reserved</i>
<i>MBOX_ERR_MULTI_OPEN</i>	<i>Error when the size of virtual device slot is too big.</i>
<i>MBOX_ERR_DEV_NOT_OPENED</i>	<i>Error when device is not opened.</i>
<i>MBOX_ERR_DEV_NOT_FOUND</i>	<i>Error when device is not found.</i>
<i>MBOX_ERR_OS_INIT</i>	<i>Reserved</i>
<i>MBOX_ERR_SIG_INIT</i>	<i>Reserved</i>
<i>MBOX_ERR_OPP_INIT</i>	<i>Reserved</i>
<i>MBOX_ERR_IOCTL</i>	<i>Reserved</i>

Remark

None

5.3 Mailbox Device Error Defines

These error messages are caused by software virtual device of mailbox. These errors mean that there are problems to use APIs of mailbox device driver.

Declaration

```
#define MBOX_DEV_ERR_NONE          (0)
#define MBOX_DEV_ERR_FULL         (-1)
#define MBOX_DEV_ERR_EXIST        (-2)
#define MBOX_DEV_ERR_INVALID_CH   (-3)
#define MBOX_DEV_ERR_HASH_MAP_FULL (-4)
#define MBOX_DEV_ERR_INVALID_NAME (-5)
#define MBOX_DEV_ERR_KEY_NOTINUSED (-6)
#define MBOX_DEV_ERR_DEV_NULL     (-7)
#define MBOX_DEV_GET_AGAIN        (0xFFFFFFFFFUL)
```

Description

Value	Description
<i>MBOX_DEV_ERR_NONE</i>	<i>For success return</i>
<i>MBOX_DEV_ERR_FULL</i>	<i>Reserved</i>
<i>MBOX_DEV_ERR_EXIST</i>	<i>Reserved</i>
<i>MBOX_DEV_ERR_INVALID_CH</i>	<i>Error when invalid channel is passed into driver API.</i>
<i>MBOX_DEV_ERR_HASH_MAP_FULL</i>	<i>Error when every virtual device slot is in use.</i>
<i>MBOX_DEV_ERR_INVALID_NAME</i>	<i>Error when device name is null.</i>
<i>MBOX_DEV_ERR_KEY_NOTINUSED</i>	<i>Error when device name does not exist in virtual device slot.</i>
<i>MBOX_DEV_ERR_DEV_NULL</i>	<i>Error when null device is passed.</i>
<i>MBOX_DEV_GET_AGAIN</i>	<i>Error when recv API needs to be called again.</i>

Remark

None

5.4 Others

Mailbox driver provides and opens the maximum size of hardware and software resources to send and receive data using mailbox driver. Some of definitions can be modified, others cannot, but the size of mailbox hardware register must be fixed.

Declaration

```
#define MBOX_CMD_FIFO_SIZE    (8UL)
#define MBOX_DATA_FIFO_SIZE  (128UL)
#define MBOX_MAX_QUEUE_SIZE  (4)
#define MBOX_MAX_MULTI_OPEN  (10UL)
#define MBOX_MAX_KEY_SIZE    (6)
```

Description

Value	Description
<i>MBOX_CMD_FIFO_SIZE</i>	<i>Word size of command buffer to be sent/received. (fixed)</i>
<i>MBOX_DATA_FIFO_SIZE</i>	<i>Word size of data buffer to be sent/received. (fixed)</i>
<i>MBOX_MAX_QUEUE_SIZE</i>	<i>The maximum size of software buffer for user to read. (modifiable)</i>
<i>MBOX_MAX_MULTI_OPEN</i>	<i>The maximum number of virtual devices per one channel. (modifiable)</i>
<i>MBOX_MAX_KEY_SIZE</i>	<i>The maximum string length of virtual device name. (fixed)</i>

Remark

The size of word is 4 bytes.

6 STRUCTURES

6.1 MBOXCallback_t

This structure contains function pointers to be called with the mailbox driver. The mailbox driver controls blocking mode of data transmission using the function pointers below. This structure also makes mailbox driver APIs thread safe and guarantees API's reentrancy.

Declaration

```
typedef struct __MBOX_CALLBACK_FUNC__  
{  
    MBOXInitFunc    osInit;  
    MBOXInitFunc    rsInit;  
    MBOXInitFunc    wsInit;  
    MBOXCaLLFunc    wrLock;  
    MBOXCaLLFunc    wrUnlk;  
    MBOXCaLLFunc    rdWake;  
    MBOXCaLLFunc    rdWait;  
    MBOXCaLLFunc    wrWake;  
    MBOXCaLLFunc    wrWait;  
    MBOXCaLLFunc    wrWclr;  
} MBOXCallback_t;
```

Description

Value	Description
<i>osInit</i>	<i>A Lock such as Mutex or Semaphore must be created in this function to be used in wrLock. And unique ID must be stored at passed parameter (*id).</i>
<i>rsInit</i>	<i>A Signal must be created in this function to be used in rdWake and rdWait for a blocking receive. And unique ID must be stored at parameter (*id) passed.</i>
<i>wsInit</i>	<i>A signal must be created in this function to be used in wrWake and wrWait for a blocking send. And unique ID must be stored at parameter (*id) passed.</i>
<i>wrLock</i>	<i>Mutex or Semaphore must be obtained in this function for thread safe of mailbox transmission.</i>
<i>wrUnlk</i>	<i>Mutex or Semaphore must be released in this function for mailbox send.</i>
<i>rdWake</i>	<i>This function must send a signal for a blocking receive. This function is called at interrupt service routine for each open device.</i>
<i>rdWait</i>	<i>This function must wait a signal for data to be received. This function is called at read API when the queue of an open device is empty.</i>
<i>wrWake</i>	<i>This function must send a signal for a blocking send. This function is called at interrupt service routine for each open device.</i>
<i>wrWait</i>	<i>This function must wait a signal for which data to be sent is read by other cores. This function is called at write API after sending data.</i>
<i>wrWclr</i>	<i>This function must clear a signal for which data to be sent is read by other cores. This function is called at write API before sending data.</i>

Remark

None

6.2 MBOXMsage_t

This structure is an element of user buffer (queue). The user buffer is written by interrupt for received data and read by driver APIs for *MBOX_DevRecvXXX*. All information in this structure is transmitted to the user by deep copy into parameters.

Declaration

```
typedef struct __MBOX_MESSAGE__  
{  
    Uint32 mmDLen;  
    Uint32 mmCmnd[MBOX_CMD_FIFO_SIZE];  
    Uint32 mmData[MBOX_DATA_FIFO_SIZE];  
} MBOXMsage_t;
```

Description

Value	Description
<i>mmDLen</i>	<i>Size of received Data FIFO</i>
<i>mmCmnd</i>	<i>Received data from mailbox command FIFO</i>
<i>mmData</i>	<i>Received data from mailbox data FIFO</i>

Remark

Do not change this structure.

6.3 MBOXQueue_t

This structure represents a queue which is software buffer for received data in mailbox. The ***MBOX_MAX_QUEUE_SIZE*** is modifiable by the user in accordance with system capabilities.

Declaration

```
typedef struct __MBOX_QUEUE__  
{  
    Int32      qiFrnt;  
    Int32      qiRear;  
    MBOXMsage_t qiData[MBOX_MAX_QUEUE_SIZE];  
} MBOXQueue_t;
```

Description

Value	Description
<i>qiFrnt</i>	Front index of user buffer (queue)
<i>qiRear</i>	Rear index of user buffer (queue)
<i>qiData</i>	Received Data for user. Refer to the MBOXMsage t .

Remark

Do not change this structure.

6.4 MBOXDvice_t

This structure represents a virtual device for mailbox channel.

Declaration

```
typedef struct __MBOX_DEVICE__
{
    MBOXCh_t      dvChnl;
    Int32         dvIden;
    UInt8         dvFlag;
    UInt32        dvSiid;
    UInt8         dvName[MBOX_MAX_KEY_SIZE + 1];
    MBOXQueue_t * dvQueue;
} MBOXDvice_t;
```

Description

Value	Description
<i>dvChnl</i>	Mailbox Channel: <i>MBOXCh_t</i> . Refer to the MBOXCh t .
<i>dvIden</i>	Mailbox Virtual Device Identification
<i>dvFlag</i>	Reserved
<i>dvSiid</i>	Signal Identification for Blocking Receive
<i>dvName</i>	Mailbox Virtual Device Name
<i>dvQueue</i>	Software Queue Pointer for Received Data. Refer to the MBOXQueue t .

Remark

Do not change this structure returned by *MBOX_DevOpen* API.

7 CALLBACK FUNCTIONS

7.1 MBOXInitFunc

This type is defined for initializing operating system resources. Refer to *MBOX_OsInit*, *MBOX_RsInit* and *MBOX_WsInit* at *mbox.c*.

Declaration

```
typedef int32 (*MBOXInitFunc)
(
    uint32 *   id
);
```

Parameters

Value	Description
<i>id</i>	<i>Identifiers for Operating System resource</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success on System Adaption Layer</i>
<i>SAL_RET_FAIL</i>	<i>Error on System Adaption Layer</i>

Remark

None

7.2 MBOXCallFunc

This type is defined for implementing operating system resources. Refer to *MBOX_WrLock*, *MBOX_WrUnlk*, *MBOX_RdWake*, *MBOX_RdWait*, *MBOX_WrWake*, *MBOX_WrWclr* and *MBOX_WrWait* at *mbox.c*.

Declaration

```
typedef int32 (*MBOXCaLLFunc)
(
    uint32 id
);
```

Parameters

Value	Description
<i>id</i>	<i>Identifiers for Operating System resource</i>

Return

Value	Description
<i>MBOX_SUCCESS</i>	<i>Success on System Adaption Layer</i>
<i>MBOX_ERR_TIMEOUT</i>	<i>A value must be returned on timeout.</i>
<i>SAL_RET_FAIL</i>	<i>Error on System Adaption Layer</i>

Remark

None

8 APIs

8.1 MBOX_Init

This function initializes mailbox hardware devices and sets callback functions for operating system resources. This API can be modified by users for their own purpose.

Declaration

```
void MBOX_Init  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

8.2 MBOX_DevInit

This function initializes the mailbox hardware device. Other APIs are valid after initializing mailbox hardware using this API.

Declaration

```
int32 MBOX_DevInit
(
    MBOXCh_t      siCh,
    uint32        uiIlv,
    uint32        ucOpn,
    MBOXCallback_t * pFunc
);
```

Parameters

Value	Description
<i>siCh</i>	A Mailbox channel to be initialized. It must be one of MBoxCh_t. Refer to the MBOXCh_t .
<i>uiIlv</i>	Fixed to MBOX_INT_LEVE_8.
<i>ucOpn</i>	The number of virtual devices to be created on this mailbox channel
<i>pFunc</i>	Structure for callback functions. Refer to the MBOXCallback_t .

Return

Value	Description
MBOX_DEV_ERR_NONE	Mailbox hardware device is properly initialized.
MBOX_DEV_MULTI_OPEN	<i>ucOpn</i> parameter exceeds MBOX_MULTI_OPENED
MBOX_DEV_ERR_ARGUMENT	<i>siCh</i> parameter exceeds MBOX_CH_MAX

Remark

None

8.3 MBOX_DevOpen

This function creates a virtual device to use hardware mailbox device. The return value is used as a parameter of send/receive APIs of mailbox device driver.

Declaration

```
MBOXDvice_t* MBOX_DevOpen
(
    MBOXCh_t    siCh,
    int8 *      pcDevName,
    uint8       ucFlag
);
```

Parameters

Value	Description
siCh	A Mailbox channel to be initialized. It must be one of MBoxCh_t. Refer to the MBOXCh_t .
pcDevName	$0 < \text{String Length of pcDevName} \leq 6$ A pointer of 7 bytes length character array with null termination . Example: A[7] = {'A', 'B', 'C', 'D', 'E', 'F', NULL} A pointer of 7 bytes length character array with null padding, unless error will occur . Example: A[7] = {'A', 'B', 'C', NULL, NULL, NULL, NULL}
uint8	Reserved

Return

Value	Description
MBOXDvice_t *	A pointer of mailbox virtual device
NULL	Failure

Remark

The size of word is 4 bytes.

8.4 MBOX_DevSend

This function sends command (6 words) and data (128 words) to an open mailbox device.

Declaration

```
int32 MBOX_DevSend
(
    MBOXDvice_t * pDev,
    uint32 *      puiCmd,
    uint32 *      puiData,
    Uint32        uilen
);
```

Parameters

Value	Description
<i>pDev</i>	A pointer of an open mailbox device
<i>puiCmd</i>	A pointer of mailbox command to be sent. It must be 6 words.
<i>puiData</i>	A pointer of mailbox data to be sent. It must be within 128 words.
<i>uilen</i>	A length of mailbox data. It must be greater than or equal to 0.

Return

Value	Description
<i>MBOX_SUCCESS</i>	Success
<i>MBOX_ERR_ARGUMENT</i>	Validation failure for <i>pDev</i> , <i>puiData</i> , and <i>uilen</i>
<i>MBOX_ERR_DEV_NOT_OPENED</i>	Mailbox device passed as parameter is not opened.
<i>MBOX_ERR_FIFO_FULL</i>	Command or data FIFO is not empty. This error is returned on non-block writing. In case of non-block writing, you must retry or flush mailbox FIFO.
<i>MBOX_ERR_WRITE</i>	Mailbox hardware is not initialized or de-initialized suddenly.

Remark

The size of word is 4 bytes.

8.5 MBOX_DevSendSss

This function sends command (8 words) and data (128 words) to mailbox device for an open HSM channel. Do not use this API with mailbox device opened with other channels. This API may block and stop entire system with mailbox device using Non-HSM channel.

Declaration

```
int32 MBOX_DevSendSss
(
    MBOXDvice_t * pDev,
    uint32 *      puiCmd,
    uint32 *      puiData,
    Uint32        uiLen
);
```

Parameters

Value	Description
<i>pDev</i>	A pointer of an open mailbox device
<i>puiCmd</i>	A pointer of mailbox command to be sent. It must be 6 words.
<i>puiData</i>	A pointer of mailbox data to be sent. It must be within 128 words.
<i>uiLen</i>	A length of mailbox data. It must be greater and equal than 0.

Return

Value	Description
<i>MBOX_SUCCESS</i>	On success
<i>MBOX_ERR_ARGUMENT</i>	Validation failed for <i>pDev</i> , <i>puiData</i> and <i>uiLen</i>
<i>MBOX_ERR_DEV_NOT_OPENED</i>	Mailbox device passed as parameter is not opened.
<i>MBOX_ERR_FIFO_FULL</i>	Command or data FIFO is not empty. This error returns on non-block writing. In case of non-block writing, you must retry or flush mailbox FIFO.
<i>MBOX_ERR_WRITE</i>	Mailbox hardware is not initialized or de-initialized suddenly.

Remark

The size of word is 4 bytes.

8.6 MBOX_DevSendCmd

This function sends command (6 words) to an open mailbox device.

Declaration

```
int32 MBOX_DevSendCmd
(
    MBOXDvice_t * pDev,
    uint32 *      puiCmd
);
```

Parameters

Value	Description
<i>pDev</i>	<i>A pointer of an open mailbox device</i>
<i>puiCmd</i>	<i>A pointer of mailbox command to be sent. It must be 6 words.</i>

Return

Value	Description
<i>MBOX_SUCCESS</i>	<i>On success</i>
<i>MBOX_ERR_ARGUMENT</i>	<i>Validation failed for pDev and puiCmd</i>
<i>MBOX_ERR_DEV_NOT_OPENED</i>	<i>Mailbox device passed as parameter is not opened.</i>
<i>MBOX_ERR_FIFO_FULL</i>	<i>Command or data FIFO is not empty. This error returns on non-block writing. In case of non-block writing, you must retry or flush mailbox FIFO.</i>
<i>MBOX_ERR_WRITE</i>	<i>Mailbox hardware is not initialized or de-initialized suddenly.</i>

Remark

The size of word is 4 bytes.

8.7 MBOX_DevSendData

This function sends data (128 words) to an open mailbox device.

Declaration

```
int32 MBOX_DevSendData
(
    MBOXDvice_t * pDev,
    uint32 *      puiData,
    Uint32        uiLen
);
```

Parameters

Value	Description
<i>pDev</i>	A pointer of an open mailbox device.
<i>puiData</i>	A pointer of mailbox data to be sent. It must be within 128 words.
<i>uiLen</i>	A length of mailbox data. It must be greater and equal than 0.

Return

Value	Description
<i>MBOX_SUCCESS</i>	On success
<i>MBOX_ERR_ARGUMENT</i>	Validation failed for <i>pDev</i> , <i>puiData</i> and <i>uiLen</i>
<i>MBOX_ERR_DEV_NOT_OPENED</i>	Mailbox device passed as parameter is not opened.
<i>MBOX_ERR_FIFO_FULL</i>	Command or data FIFO is not empty. This error returns on non-block writing. In case of non-block writing, you must retry or flush mailbox FIFO.
<i>MBOX_ERR_WRITE</i>	Mailbox hardware is not initialized or de-initialized suddenly.

Remark

The size of word is 4 bytes.

8.8 MBOX_DevRecv

This function receives command (6 words) and data (128 words) from the open mailbox device.

Declaration

```
int32 MBOX_DevRecv
(
    MBOXDvice_t * pDev,
    uint32 *      puiCmd,
    uint32 *      puiData
);
```

Parameters

Value	Description
<i>pDev</i>	A pointer of an open mailbox device.
<i>puiCmd</i>	A pointer of buffer to receive mailbox command. It must be 6 words.
<i>puiData</i>	A pointer of buffer to receive mailbox data. It must be 128 words.

Return

Value	Description
≥ 0	A length of <i>puiData</i> . If 0, only <i>puiCmd</i> (6 words) is received.
<i>MBOX_ERR_DEV_NOT_OPENED</i>	Mailbox device passed as parameter is not opened.
<i>MBOX_ERR_ARGUMENT</i>	Validation failed for <i>pDev</i> , <i>puiCmd</i> and <i>puiData</i> .

Remark

The size of word is 4 bytes.

8.9 MBOX_DevRecvSss

This function receives command (8 words) and data (128 words) from mailbox device for an open HSM channel. Do not use this API with mailbox device opened with other channels. This API may block and stop entire system with mailbox device using Non-HSM channel.

Declaration

```
int32 MBOX_DevRecvSss
(
    MBOXDvice_t * pDev,
    uint32 *      puiCmd,
    uint32 *      puiData
);
```

Parameters

Value	Description
<i>pDev</i>	A pointer of an open mailbox device.
<i>puiCmd</i>	A pointer of buffer to receive mailbox command. It must be 8 words.
<i>puiData</i>	A pointer of buffer to receive mailbox data. It must be 128 words.

Return

Value	Description
≥ 0	A length of <i>puiData</i> . If 0, only <i>puiCmd</i> (6 words) is received.
<i>MBOX_ERR_DEV_NOT_OPENED</i>	Mailbox device passed as parameter is not opened.
<i>MBOX_ERR_ARGUMENT</i>	Validation failed for <i>pDev</i> , <i>puiCmd</i> and <i>puiData</i> .

Remark

The size of word is 4 bytes.

8.10 MBOX_DevRecvCmd

This function receives command (6 words) from the open mailbox device.

Declaration

```
int32 MBOX_DevRecvCmd
(
    MBOXDvice_t * pDev,
    uint32 *      puiCmd
);
```

Parameters

Value	Description
<i>pDev</i>	<i>A pointer of an open mailbox device</i>
<i>puiCmd</i>	<i>A pointer of buffer to receive mailbox command. It must be 6 words.</i>

Return

Value	Description
0	<i>puiCmd (6 words) is received.</i>
<i>MBOX_ERR_DEV_NOT_OPENED</i>	<i>Mailbox device passed as parameter is not opened.</i>
<i>MBOX_ERR_ARGUMENT</i>	<i>Validation failed for pDev, puiCmd.</i>

Remark

The size of word is 4 bytes.

8.11 MBOX_DevRecvData

This function receives data (128 words) from the open mailbox device.

Declaration

```
int32 MBOX_DevRecvData
(
    MBOXDvice_t * pDev,
    uint32 *      puiData
);
```

Parameters

Value	Description
<i>pDev</i>	A pointer of virtual device created by <i>MBOX_DevOpen</i> in Chapter 8.3.
<i>puiData</i>	A pointer of data buffer of which the data received will be stored.

Return

Value	Description
≥ 0	A length of <i>puiData</i> .
<i>MBOX_ERR_DEV_NOT_OPENED</i>	Mailbox device passed as parameter is not opened.
<i>MBOX_ERR_ARGUMENT</i>	Validation failed for <i>pDev</i> , and <i>puiData</i>

Remark

The size of word is 4 bytes.

8.12 MBOX_DevIoctl

This function controls mailbox hardware and checks mailbox status.

Declaration

```
int32 MBOX_DevIoctl
(
    MBOXDvice_t * pDev,
    uint32      uiCmd,
    uint32 *     puiValue
);
```

Parameters

Value	Description
<i>pDev</i>	A pointer of virtual device created by <i>MBOX_DevOpen</i> in Chapter 8.3.
<i>uiCmd</i>	Mailbox I/O Command. Refer to Chapter 5.1 Mailbox I/O Commands
<i>puiValue</i>	Value to be read or written.

Return

Value	Description
<i>MBOX_SUCCESS</i>	Success
<i>MBOX_ERR_ARGUMENT</i>	Failure

Remark

None

9 HOW TO USE MAILBOX DRIVER

9.1 Echo Server Example: Blocking Read/Write

```
uint32 mCmd[MBOX_CMD_FIFO_SIZE - 2] = {0, };
uint32 mData[MBOX_DATA_FIFO_SIZE] = {0, };

static void Example_Echo_Server_Task(void *pArg)
{
    int32      ret = -1;
    MBOXDvice_t * dev = NULL;

    if ((dev = MBOX_DevOpen(MBOX_CH_CA72MP_NONSECURE, "ECHOE1\0", 0)) != MBOX_SUCCESS)
        goto err;

    while (1)
    {
        SAL_MemSet(mCmd, 0x00, MBOX_CMD_FIFO_SIZE - 2);
        SAL_MemSet(mData, 0x00, MBOX_DATA_FIFO_SIZE);

        if((ret = MBOX_DevRecv(dev, mCmd, mData)) > 0)
        {
            if ((ret = MBOX_DevSend(dev, mCmd, mData)) != MBOX_SUCCESS) // wait until sent data is read
            {
                print write err;
            }
        }
        else
        {
            if (ret == 0)
                print mCmd "only command has been received."
            else
                print read err;
        }
    }
}

err:
    print task removed.
}
```

9.2 Echo Client Example: Non-blocking Read/Write

```

uint32 mCmd[MBOX_CMD_FIFO_SIZE - 2] = {0, };
uint32 mData[MBOX_DATA_FIFO_SIZE] = {0, };

static void Example_Echo_Client_Task(void *pArg)
{
    int32      ret = -1;
    MBOXDvice_t * dev = NULL;

    if ((dev = MBOX_DevOpen(MBOX_CH_CA72MP_NONSECURE, "ECHOE2\0", 0)) != MBOX_SUCCESS)
    {
        goto err;
    }

    while (1)
    {
        set data to mCmd within 6 words;
        set data to mData within 128 words;

        if((ret = MBOX_DevSend(dev, mCmd, mData)) != MBOX_SUCCESS) // return immediately after sending
        {
            SAL_TaskSleep(100);
            continue;
        }

        SAL_MemSet(mCmd, 0x00, MBOX_CMD_FIFO_SIZE - 2);
        SAL_MemSet(mData, 0x00, MBOX_DATA_FIFO_SIZE);

        while (1)
        {
            if ((ret = MBOX_DevRecv(dev, mCmd, mData)) >= 0)
            {
                check if received mCmd and mData is same as sent.
                break;
            }

            SAL_TaskSleep(100);
        }

        SAL_TaskSleep(1000);
    }

err:
    print task removed.
}

```

9.3 Emergency Message Transmission Example: Blocking Read/Write

```

uint32 mCmdReq[MBOX_CMD_FIFO_SIZE - 2] = {"SHUT", "DOWN", "REQ", "FROM", "MCU", "BSP"};
uint32 mCmdRes[MBOX_CMD_FIFO_SIZE - 2] = {0, };

static void Example_Shutdown_Sending_Task
(
    void *pArg
)
{
    int32 ret = -1;
    // refer to "18. Mailbox" of the "PART 11-MICOM SUB-SYSTEM" in the "TCC805x Full Specification [1]"
    uint32 uiVal = (MBOX_CMD_FLUSH_BIT|MBOX_DAT_FLUSH_BIT);
    MBOXDvice_t *dev = NULL;

    if ((dev = MBOX_DevOpen(MBOX_CH_CA72MP_NONSECURE, "ECHOE2\0", 0)) != MBOX_SUCCESS)
        goto err;

    while (1)
    {
        read shunt down key(); // blocked and return if key is read.

        // flush every data to be sent before
        if ((ret = MBOX_DevIoctl(dev, MBOX_IO_CMD_SET_CTRL_REG, &uiVal)) != MBOX_SUCCESS)
        {
            SAL_TaskSleep(100);
            continue;
        }

        if((ret = MBOX_DevSendCmd(dev, mCmdReq)) != MBOX_SUCCESS) // wait until sent data is read
        {
            SAL_TaskSleep(100);
            continue;
        }

        SAL_MemSet(mCmd, 0x00, MBOX_CMD_FIFO_SIZE - 2);

        if ((ret = MBOX_DevRecvCmd(dev, mCmdRes)) >= 0)
        {
            check if Res is Ok.
            if OK, then break;
        }

        SAL_TaskSleep(100);
    }

err:
    print task removal.
    system restart;
}

```

9.4 Checking Peer Mailbox Ready Example

```
int32 Checking_Peer_Mailbox_Ready(MBOXDvice_t *dev)
{
    int32 ret          = IPC_ERR_COMMON;
    uint32 oppinit      = 0;

    while (oppinit == 0)
    {
        if ((ret = MBOX_DevIoctl(dev, MBOX_IO_CMD_GET_OPP_STS, &oppinit)) == MBOX_SUCCESS)
        {
            if (oppinit == 0x01U)
                break;
        }

        Check_Escape_Condition(Timeout);
        SAL_TaskSleep(100);
    }

    return ret;
}
```

9.5 Communication With U-Boot Layer of AP

A mailbox virtual device is open and reserved for use by each mailbox physical channel. The mailbox virtual device is open as "BCTRL\0" and used to communicate with U-Boot layer of AP. A device handler is stored in spMboxDev which is external global variable.

```
static void BCTRL_TaskRun
(
    uint32                uiCore,
    MBOXCh_t              siCh
)
{
    MBOXDvice_t *BCTRLMboxDev;

    BCTRLMboxDev = spMboxDev[(uint32) siCh].mboxDev; // get reserved mailbox device for u-boot
    while(BCTRLMboxDev != NULL_PTR)
    {
        SAL_MemSet(&BCTRLCmd[uiCore][0], 0x00, MBOX_CMD_FIFO_SIZE);
        if (MBOX_DevRecvCmd(BCTRLMboxDev, &BCTRLCmd[uiCore][0]) >= 0) // receive command from u-boot
        {
            // Check ID From BL0, 1, 2, 3.
            if (BCTRLCmd[uiCore][0] == BCTRL_ID_BOOT_SEQUENCE)
            {
                if (BCTRL_CheckStatus(uiCore, &BCTRLCmd[uiCore][0]) == BCTRL_STAT_READY)
                {
                    BCTRLCmd[uiCore][1] = BCTRL_COMMAND_GO;
                    BCTRLCmd[uiCore][2] = BCTRL_SignalDramInit(uiCore);

                    MBOX_DevSendCmd(BCTRLMboxDev, &BCTRLCmd[uiCore][0]); // send command to u-boot
                    BCTRLStat[uiCore] = BCTRL_STAT_GO;
                    mcu_printf("[BCTRL] ch %d status %d\n", (int32) siCh, (int32) BCTRLStat[uiCore]);
                    PMIO_APBootOk();
                }
                else
                {
                    mcu_printf("[E][BCTRL] ch %d status %d\n", (int32) siCh, (int32)
BCTRLStat[uiCore]);
                }
            }
            else
            {
                mcu_printf("[E][BCTRL] ch %d Unused BCTRL ID %d\n", (int32) siCh, (int32)
BCTRLStat[uiCore]);
            }
        }
    }

    mcu_printf("[E][BCTRL] terminated %d", (int32) siCh);
}
```

Note: Refer to "sources/app.sample/app.boot.ctrl/bootctrl.c".

10 REFERENCES

- [1] TCC805x Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

11 REVISION HISTORY

Rev. 1.01: 2021-06-08

- Updated
 - Chapter 8.10: Return
 - Chapter 8.11: Parameter and return
- Added
 - Chapter 9.5: Communication With U-boot Layer of AP

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."